Digital Object Identifier

Evaluating Genetic Algorithms for Compiler Pass Ordering and Selection in a Small Three-Address-Code Optimizer

Aditya Rawat, Abhishek Kumar, Tamanna Thakur, and Sanvi Tiwari
Department of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India

Corresponding author: Aditya Rawat (Aditya.jaikrishna2024@vit.student.ac.in).

This work received no external funding. Author contact emails: Aditya Rawat, Aditya.jaikrishna2024@vit.student.ac.in; Abhishek Kumar, abhishek.kumar2024c@vitstudent.ac.in; Tamanna Thakur, tamanna.thakur2023@vitstudent.ac.in; Sanvi Tiwari, Sanvi.tiwari2024@vitstudent.ac.in. Author biographies, ORCID identifiers, and portrait photographs remain to be supplied by the authors.

ABSTRACT Compiler optimization quality depends partly on the order and selection of transformations, but a search method must be evaluated against meaningful benchmarks and strong baselines. This paper revisits a small three-address-code (3AC) optimizer after identifying that its original six-pass permutation space contains only 720 candidates and is exhaustively searchable. The revised prototype adds variable-length sequences that permit passes to be omitted or repeated, a maximum sequence length of 12, an input instruction for straight-line input-driven programs, fixed-point pipeline execution, and finite-test output validation inside candidate scoring. Seven hand-authored benchmarks are evaluated. Across 20 seeds, the fixed-length genetic algorithm reaches the exact single-pass static-cost optimum in all 140 benchmark-seed runs, generally by generation one; this confirms that the original problem is too small to justify a genetic algorithm in practice. For the enlarged variable-length search, the genetic algorithm and random search receive 750 candidate evaluations per seed. The genetic algorithm obtains lower median cost on two constant-heavy benchmarks and ties random search on five; it does not improve on the fixed-point baseline. Exhaustive fixed-point evaluation shows that, for this pass set and benchmark suite, all 720 orderings converge to the same final cost per benchmark, eliminating ordering-based ranking under that policy. Leave-one-pass-out ablation and weight-sensitivity results are also reported. The study remains a transparent small-IR case study: branches, loops, native runtime, binary size, and formal equivalence are not evaluated. No claim of native speedup or general genetic-algorithm superiority is made.

INDEX TERMS Compiler optimization, phase ordering, pass selection, genetic algorithms, variable-length chromosomes, fixed-point optimization, translation validation, static cost model.

I. INTRODUCTION

Compilers transform source programs into intermediate representations (IRs), apply analyses and transformations, and generate target code. Individual optimization passes are well understood in isolation, yet their combined effect can depend on both the order in which they execute and whether a pass is repeated. Constant propagation may expose constants for folding; algebraic simplification can replace a binary operation with a copy; copy propagation can then expose redundant expressions; and dead-code elimination (DCE) can remove computations that no longer contribute to observable output. A pass applied too early may miss an opportunity that becomes visible only after another pass has run.

The compiler phase-ordering problem asks which sequence of transformations should be applied to a program. Pass selection asks which transformations should be enabled, while phase ordering additionally considers their order. In practical compilers, the space can be enormous because passes may be optional, repeated, parameterized, or constrained by dependencies. Search is useful only when its effectiveness is assessed against realistic objectives and baselines. A genetic algorithm (GA) can explore discrete sequences, but stochastic search alone does not establish either optimality or superiority over simpler methods.

The initial version of this project represented a candidate as a permutation of six passes: constant propagation (CP), constant folding (CF), copy propagation (CopyP), common-subexpression elimination (CSE), algebraic simplification

(AS), and DCE. Because each pass appeared exactly once, the search space had $6! = 720$ candidates. Exhaustive enumeration took approximately 0.12 s across the original five constant-only benchmarks, and a random search with 750 draws found the same best cost on every benchmark in the recorded run. The original experiment therefore demonstrated the limitations of the setup more clearly than it demonstrated a need for a GA.

This revision adds optional and repeated passes through variable-length chromosomes, input-driven straight-line programs, a fixed-point mode, candidate-level output checks, 20-seed experiments, a matched-budget random baseline, pass ablation, and cost-weight sensitivity. The study is deliberately scoped: the IR supports INPUT, CONST, COPY, BINOP, and PRINT instructions, but not branches, loops, memory operations, calls, or a control-flow graph. The benchmark suite is broader than the original one but is not a standard benchmark suite. Native code generation and performance measurement remain outside the implementation.

The contributions are as follows:

1) We document why the original six-pass permutation problem is an unsuitable setting for claiming GA necessity, using exhaustive enumeration and 20 independent seeds.
2) We evaluate a variable-length representation that allows omission and repetition of passes, capped at 12 passes per candidate.
3) We include finite-test semantic validation in candidate scoring, so invalid candidates cannot win by reducing the static score.
4) We compare GA and random search under the same 750-candidate budget and report a fixed-point cleanup baseline, exhaustive cost distributions, pass ablation, and weight sensitivity.
5) We provide reproducible scripts and machine-readable results while distinguishing the measured static objective from runtime and executable-size performance.

The main conclusion is intentionally qualified. The fixed-length GA reaches exact optima quickly because the search space is tiny. The variable-length GA improves on random search for two of the constant-heavy benchmarks but ties it on the remaining five and does not outperform the fixed-point baseline. These findings motivate larger and more realistic follow-up experiments rather than a claim of GA superiority.

II. BACKGROUND AND RELATED WORK

A. SEARCH-BASED OPTIMIZATION AND PHASE ORDERING

Cooper, Schielke, and Subramanian used genetic algorithms to reduce code space, demonstrating that optimization sequences can be searched rather than accepted only from a default pipeline [1]. Pan and Eigenmann developed combined-elimination orchestration and evaluated compiler choices against measured performance on SPEC CPU2000, reporting that a search can approach exhaustive-search quality with lower tuning effort [2]. Hoste and Eeckhout's COLE framework explored optimization levels as a multi-objective problem, including trade-offs among code quality, size, energy, and compilation cost [3]. These studies use production compiler settings and objectives that differ from the static tuple-count score in the present project.

Machine-learning-assisted approaches use program features, historical measurements, or learned policies to guide optimization. Milepost GCC investigated machine-learning-enabled self-tuning compilation [5], while Ashouri et al. survey compiler autotuning with machine learning and identify the importance of training data, objective choice, and evaluation methodology [4]. Liang et al. use a coreset and normalized value prediction to learn compiler pass orders for new programs [6]. These methods address a broader search and generalization problem than the independent per-benchmark GA used here.

Shackleton is especially relevant because it applies linear genetic programming to LLVM optimization-pass sequences. Its study discusses framework choices and evaluates two applications, reporting both strengths and limitations of sequence search [7]. Cooper, Subramanian, and Torczon also discuss adaptive optimizing compilers and the broader role of adaptation in compiler design [15]. The present implementation is much smaller: its passes operate on a custom 3AC IR and its objective is static cost. Thus, Shackleton is a methodological comparator, not a directly comparable performance baseline.

Recent work increasingly incorporates reinforcement learning (RL), pass-dependence information, and large language models (LLMs). AutoPhase combines program features with deep reinforcement learning for LLVM-based high-level synthesis and reports circuit-performance improvements against LLVM -O3 in its target setting [8]. Liu et al. model pass dependencies and group related passes into subsequences, evaluating the method on CBench and PolyBench and reporting runtime and code-size gains in their own environment [9]. DeCOS combines data-efficient RL and LLM assistance for compiler optimization selection [10]. REASONING COMPILER uses LLM-generated proposals with structured Monte Carlo tree search for model-serving workloads [11]. These results motivate structured search and richer objectives, but their numbers cannot be transferred to this custom IR or compared with its static cost units.

Jantz and Kulkarni's study of optimization phase selection during dynamic JIT compilation appeared at VEE 2013, pages 131–142, with DOI 10.1145/2451512.2451539 [12]. It is relevant to the distinction between selecting phases and ordering them, and to the possibility that a large theoretical space does not guarantee that a heuristic will realize performance gains. Queiroz Junior, da Silva, and Rodriguez's hybrid approach combines machine learning and iterative compilation and reports results against LLVM optimization levels in its setting [13]. CompilerGym provides a programmatic environment for compiler optimization research, including learning-based methods [14].

B. STRUCTURED COMPARISON OF RELATED WORK

Table 1 summarizes selected studies by method, evaluation, and the gap relative to the present study. The reported outcomes are paraphrased from the cited publications and are not directly compared numerically because their compilers, benchmarks, hardware, and objective functions differ.

C. RESEARCH GAP

The gap addressed here is not the absence of compiler phase-ordering research. Rather, it is the need for a transparent experimental demonstration that makes search-space size, baseline strength, correctness checks, random-seed variability, and objective limitations explicit in a small educational optimizer. The project is not intended to replace LLVM-based work or to claim better performance than established approaches. Its value is that each candidate can be inspected and the fixed-length search space can be completely enumerated. The expanded variable-length experiments provide an initial test of whether the GA has value when sequences can omit or repeat passes; the results show only limited gains against random search and no gain against fixed-point cleanup.

III. INTERMEDIATE REPRESENTATION AND OPTIMIZATION PASSES

A. SUPPORTED IR

The optimizer uses a list of instruction tuples. The supported opcodes are INPUT, CONST, COPY, BINOP, and PRINT. BINOP supports addition, subtraction, multiplication, and division. Variables are represented by strings, constants by numeric literals, and PRINT operations are observable outputs. INPUT introduces a value supplied by the interpreter for each test case. The IR is linear: there are no branch labels, conditional jumps, loops, arrays, pointers, memory effects, function calls, or control-flow joins.

A small input-driven example is shown below:

```
INPUT x
INPUT y
t1 = x + y
t2 = t1 * 1
t3 = t2 + 0
PRINT t3
```

This example allows algebraic simplification and dead-code cleanup to be tested without replacing all values by compile-time constants. It is still not a realistic loop kernel. In particular, the input instruction models an externally supplied scalar but does not model input parsing, I/O cost, or stateful memory.

B. PASS DEFINITIONS AND INTERACTIONS

CP substitutes known constant values into later uses while tracking assignments. CF evaluates binary operations whose operands are numeric literals when the operation is defined. CopyP replaces variable uses with their known copy sources. CSE identifies repeated expressions and reuses an earlier result when operand definitions remain valid. AS applies identities such as $x + 0 \rightarrow x$, $x - 0 \rightarrow x$, $x \times 1 \rightarrow x$, and $x \times 0 \rightarrow 0$. DCE removes assignments whose values are not live at any observable output. PRINT instructions remain observable and are retained.

These transformations have enabling relationships. CP can expose numeric operands for CF. AS may replace a BINOP with a COPY or CONST, enabling CopyP and DCE. CopyP can expose equivalent expressions for CSE. CSE may replace a redundant computation with a copy, which a later CopyP or DCE can simplify. The relationships are conceptual and depend on each implementation's data structures and rewrite rules; the present experiments do not isolate every causal interaction.

Pass correctness requires attention to language semantics. The implementation avoids replacing x/x with one or $0/x$ with zero when x is a variable because the divisor may be zero. The interpreter reports division-by-zero errors rather than silently producing a value. These rules are conservative for the supported numeric semantics, but the IR does not model overflow, exceptions beyond division by zero, floating-point edge cases in a complete language, aliasing, or side effects. Results are therefore limited to the implemented IR and interpreter.

C. FIXED-LENGTH AND VARIABLE-LENGTH SEQUENCE REPRESENTATIONS

The original representation is a permutation of $P = \{CP, CF, CopyP, CSE, AS, DCE\}$. Its candidate count is

$$|\mathcal{S}_{\text{perm}}| = 6! = 720. \tag{1}$$

Every pass appears once, so this representation is pass ordering only; it cannot select a subset or repeat a transformation. The revised variable-length representation is a sequence $S = (p_1, \ldots, p_\ell)$ where each $p_i \in P$ and $1 \leq \ell \leq 12$. Passes may be omitted or repeated. If every sequence of lengths one through twelve is permitted, the raw sequence count is $\sum_{\ell=1}^{12} 6^\ell$, before considering equivalent pipelines and pass semantics. This space is far larger than 720, although many sequences may yield the same final IR. The length bound keeps the search finite and the implementation tractable; it is a design cap, not a claim about an optimal production-compiler sequence length.

IV. PIPELINE POLICIES AND GENETIC ALGORITHM

A. SINGLE-PASS AND FIXED-POINT POLICIES

In single-pass mode, the candidate sequence is applied once from left to right. In fixed-point mode, the complete sequence is reapplied until the IR stops changing or the maximum of 20 rounds is reached. The implementation records whether convergence occurred. The deterministic fixed pipeline is CP $\rightarrow$ CF $\rightarrow$ CopyP $\rightarrow$ CSE $\rightarrow$ AS $\rightarrow$ DCE, iterated to a fixed point for the stronger baseline.

Fixed-point iteration changes the problem. If one pass creates an opportunity for a pass that has already run, another round can exploit it. The ranking of one-shot orderings may therefore collapse or change after iteration. To quantify this

TABLE 1. Structured comparison with related compiler-optimization research.

Study	Method	Evaluation/result reported by the study	Difference and gap relative to this work
Cooper *et al.* [1]	Genetic search for code-space reduction	Demonstrates search over compiler optimization sequences with code-size objective.	Establishes evolutionary-search precedent; uses a different compiler and a real code-space objective.
Pan and Eigenmann [2]	Combined elimination / feedback-directed orchestration	Evaluates GCC optimization choices on SPEC CPU2000 and reports performance close to exhaustive search with reduced tuning effort.	Native performance is measured; this work has only a static IR score.
Hoste and Eeckhout (COLE) [3]	Multi-objective optimization-level exploration	Explores Pareto-optimal optimization levels and trade-offs among objectives.	Addresses optimization selection and objective trade-offs more directly than the single scalar score here.
Milepost GCC [5]	Program features and machine-learning-enabled self-tuning	Uses program features and learned models to guide compiler optimization choices.	Learns across programs; the current GA does not train a model or generalize to held-out programs.
AutoPhase [8]	Random forests plus deep RL for HLS phase ordering	Reports improvements over LLVM -O3 for synthesized circuits and generalization to generated programs.	Uses LLVM and HLS circuit metrics; this study uses straight-line 3AC and no native backend.
Shackleton [7]	Linear genetic programming for LLVM pass sequences	Evaluates optimization-sequence search on two applications and analyzes framework settings.	Closest evolutionary precedent; custom passes and static cost here are much less representative.
Liu *et al.* [9]	Pass-dependence graph and subsequence generation	Reports runtime and code-size gains using CBench/PolyBench and iterative compilation frameworks.	Models dependencies explicitly; the present implementation has a fixed six-pass registry and no dependency graph.
DeCOS [10]	Data-efficient RL with LLM assistance	Uses LLMs to support compiler optimization selection with data-efficient learning.	Uses a learned decision process; this study is an interpretable, non-learning baseline.
REASONING COMPILER [11]	LLM proposals with structured Monte Carlo tree search	Evaluates LLM-guided optimization for model-serving workloads.	Targets neural/model-serving code; current IR cannot express such workloads.
This study	Fixed-length and variable-length GA; exhaustive and random baselines	20 seeds, 750-evaluation paired search, fixed-point baseline, ablation, and weight sensitivity under a static cost.	A small reproducibility case study; control flow and native performance are not evaluated.

effect, the experiment evaluates all 720 one-use orderings in both single-pass and fixed-point modes. This is feasible only because the six-pass permutation space is small. The resulting fixed-point costs are identical across all orderings for each benchmark; the fixed-point mode therefore creates ties rather than a meaningful ordering ranking in this benchmark suite.

B. FIXED-LENGTH GA

The fixed-length GA uses a population of 25, 30 generations, order crossover, swap mutation, tournament selection of size three, and elitism of two. Each chromosome remains a valid permutation. The fixed-length experiment is repeated for seeds 0–19. Because the exact single-pass optimum is known from exhaustive enumeration, the success rate, optimality gap, and first generation at which the optimum is reached can be measured directly. Repeated seeds characterize the behavior of this particular implementation; they do not overcome the small search space or imply generalization.

C. VARIABLE-LENGTH GA

The variable-length GA starts with sequences of different lengths. Crossover combines parent subsequences; mutation can alter, insert, delete, or replace passes, after which the chromosome is truncated to the maximum length of 12. Fitness is based on final static cost after applying the candidate once. The population size is 25 and the number of generations is 30, corresponding to 750 candidate evaluations per seed when the full population is evaluated at each generation. The same seeds, 0–19, are used for paired comparison with random search. The random baseline draws 750 sequences with lengths from 1 to 12 and pass IDs sampled from the same six-pass alphabet.

The comparison budget is defined in candidate evaluations, not unique sequences. Duplicate sequences may be evaluated more than once by either method. This budget equality is useful but does not make the algorithms identical in overhead: GA additionally performs selection, crossover, and mutation. The paper reports objective quality rather than claiming a speed advantage in wall-clock time. The search also does not compare against a learned policy, simulated annealing, Bayesian optimization, or LLVM optimization levels.

D. CANDIDATE-LEVEL SEMANTIC VALIDATION

Each candidate is first applied to a copy of the input IR. The validator checks instruction structure after transformation, and the reference interpreter runs the original and transformed IR over the specified input cases. If an execution error occurs or observable PRINT outputs differ, the candidate is assigned infinite cost and zero fitness. Otherwise, its static cost is calculated. This gate is used in the fixed-length and variable-length fitness evaluation paths and in the random-search comparator.

For constant-only benchmarks, the test case is the empty

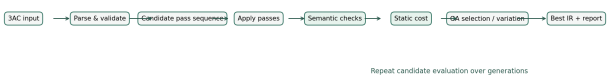


FIGURE 1. Candidate evaluation and search workflow. Semantic validation occurs before the static score is accepted.

input tuple. For input-driven benchmarks, the harness uses five finite vectors; each input position receives values drawn from zero, one, negative values, and mixed positive/negative examples. These checks are stronger than evaluating candidate syntax alone, but they are not a formal proof of equivalence. A transformation could still be wrong for an input not in the test set, and this finite-input strategy cannot validate control-flow behavior because the IR has no branches or loops.

V. COST MODEL AND EXPERIMENTAL DESIGN

A. STATIC COST

The objective is a weighted instruction count:

$$C(Q) = n_{INPUT} + n_{CONST} + n_{COPY} + 3n_{BINOP} + 2n_{PRINT}. \quad (2)$$

where n_{op} is the number of instructions of type op in transformed IR Q. The weights are project-defined units: INPUT, CONST, and COPY have weight one, BINOP weight three, and PRINT weight two. Fitness is $F(Q) = 1000/(1+C(Q))$ for semantically valid candidates and zero for invalid candidates. The monotonic transform means that maximizing fitness and minimizing cost produce the same ranking among valid candidates.

The weights were chosen to distinguish arithmetic operations from simple assignments and output operations, not estimated from hardware cycles or learned from a training set. The cost model ignores instruction latency, vectorization, memory access, register pressure, cache behavior, control-flow frequency, and target-specific lowering. A lower score should therefore be interpreted as a lower score under Eq. 2, not as faster execution or a smaller native binary.

B. WEIGHT SENSITIVITY

To test whether the ranking is unusually dependent on one hand-selected weight, each instruction weight is independently multiplied by 0.75 and 1.25, while the other weights remain unchanged. For each perturbation, costs for all 720 one-use orderings are recomputed and Spearman rank correlation with the baseline ranking is calculated. This analysis is an internal sensitivity test only. It does not validate the weights against runtime or code size, and it does not establish that the chosen cost model is a good proxy for performance. The INPUT and PRINT contributions often remain ranking-invariant because the corresponding instruction counts are constant across valid candidates for a benchmark.

C. BENCHMARKS

Seven hand-authored programs are used. BM1 exercises constant arithmetic and neutral elements. BM2 contains repeated and commutative expressions. BM3 contains a large dead computation and a small live output. BM4 exercises a chain of copies. BM5 combines neutral and absorbing algebraic identities. BM6 and BM7 add INPUT instructions and input-dependent arithmetic; BM7 also includes commutative repeated expressions. The latter two programs ensure that the evaluator cannot always replace all arithmetic with compile-time constants.

The suite is deliberately described as a constructed test suite, not a standard benchmark collection. It has no branches, loops, memory accesses, function calls, or realistic data structures. It is useful for debugging the pass interactions and correctness gate, but it is insufficient to support claims about general application performance. In particular, the constant-only cases remain easy to simplify and therefore still favor fixed-point cleanup.

D. COMPARATORS AND REPORTING METRICS

We compare four policies: (1) the unoptimized input IR, (2) the fixed pipeline applied once, (3) the fixed pipeline iterated to a fixed point, and (4) search methods. For the fixed-length problem, exhaustive enumeration supplies the exact single-pass optimum and full cost distribution. Fixed-length GA metrics include success rate, median optimality gap, and first generation at optimum across 20 seeds. For variable-length search, GA and random search use 750 evaluations per seed and are compared over 20 paired seeds. The fixed-point baseline is deterministic for a given benchmark.

The normalized optimality gap is $(C_{GA} - C^*)/\max(1, |C^*|)$, where C^* is the exact single-pass optimum. The gap is available for fixed-length candidates because exhaustive enumeration defines C^*. Variable-length candidates are not assigned an exact global optimum because the sequence space is much larger and is not exhaustively enumerated. Their comparison uses paired best costs against random search and the fixed-point baseline. All output tables and figures are generated from CSV result files by the experiment script.

E. REPRODUCIBILITY CONFIGURATION

The experiment harness uses Python and the project's pass registry. It records seeds, benchmark identifiers, sequences, costs, and convergence history. The fixed-length space is enumerated in all 720 permutations per benchmark. The fixed-length GA and variable-length GA each use seeds 0–19. Random search uses the same seed values to create paired comparisons and makes 750 draws per benchmark. The fixed-point policy has a maximum of 20 rounds. The project tests pass in the packaging environment (19 tests), but the Streamlit UI could not be launched in that environment because Streamlit was not installed; a genuine browser screenshot is therefore not included in this manuscript package.

VI. RESULTS

A. SINGLE-PASS EXHAUSTIVE SEARCH AND REPEATED-SEED GA

Table 2 summarizes the static costs. Across the seven benchmarks, the fixed-length GA reaches the exact single-pass optimum in 20 of 20 runs for every benchmark. The median optimality gap is zero for every benchmark, and the median first generation at the optimum is generation one for six benchmarks and generation 1.5 for BM5. These results are not evidence that the GA is needed: all 720 orderings can be enumerated directly, and the optimum is often reached almost immediately because many orderings have identical costs.

The exact optima occur in multiple orderings. For BM3 and BM4, one third of the 720 permutations reach the optimum. BM5 has the smallest optimal set among the constant-heavy examples, with 30 optima. BM6 and BM7 each have 120 optima. This high prevalence of ties helps explain why a random search can reach an optimum in a small budget. It also means that reporting only one selected chromosome can overstate the importance of a particular order.

The original single-pass policy leaves some obvious opportunities in intermediate output. For example, a pass that creates a simplification opportunity after the relevant cleanup pass has already executed cannot exploit it until another round. This limitation is the main motivation for measuring fixed-point execution rather than treating the one-pass optimum as a realistic final pipeline.

B. FIXED-POINT RANKING ANALYSIS

We evaluated all 720 permutations in fixed-point mode for each benchmark. All sequences converged within the 20-round guard, and no output mismatches were detected on the supplied cases. The final cost was identical across all 720 orderings for each benchmark: 2 for BM1–BM5 and 7 for BM6–BM7. Consequently, the Spearman correlation between single-pass costs and fixed-point costs is undefined because the fixed-point ranking is constant; every sequence is tied under the fixed-point objective. The single-pass optimum and fixed-point minimum should therefore not be treated as rankings of the same problem. Fixed-point iteration removes the one-pass artifact for this suite, but also makes the order choice irrelevant under these pass implementations and these benchmarks.

C. VARIABLE-LENGTH GA VERSUS RANDOM SEARCH

Table 3 compares the variable-length GA and random search over 20 paired seeds with 750 candidate evaluations per seed. The GA's median cost is lower than random search for BM1 (2 versus 6) and BM2 (2 versus 7). It ties random search on BM3, BM4, BM5, BM6, and BM7. Across 140 benchmark-seed comparisons, GA wins 40 and ties 100, with no losses. The wins are concentrated in the two constant-heavy benchmarks. On the input-driven benchmarks, both search methods reach cost 7, matching the fixed-point baseline. The variable-length GA therefore shows a limited advantage over random search on two cases but no advantage over the fixed-point baseline.

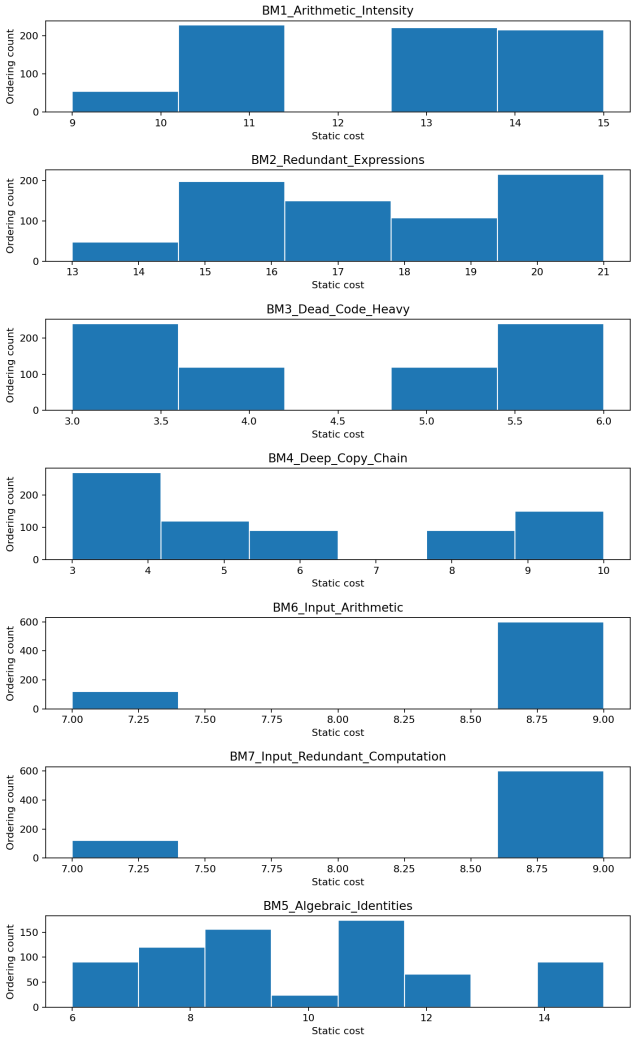


FIGURE 2. Exhaustive static-cost distributions for all 720 one-use pass orderings. These distributions describe the one-pass policy, not native execution time.

The candidate-evaluation budget is matched, but wall-clock overhead is not separately compared. The GA has genetic-operator overhead; random search simply generates and scores candidates. No native runtime or compilation time is included in the objective. The results do not establish that the GA is faster or more efficient in elapsed time, and they do not show a broad advantage across programs. The fact that fixed-point cleanup is already as good as or better than the GA is an important negative result and is retained rather than hidden.

D. PASS ABLATION

We performed a leave-one-pass-out ablation of the fixed-point pipeline. For each pass, the pass is removed from the sequence and the remaining sequence is iterated to a fixed point. Table 4 reports the cost increase relative to the all-pass

TABLE 2. Single-pass costs and fixed-point baseline (static cost units).

Benchmark	Initial	Fixed, one pass	Fixed point	Exact one-pass min.	One-pass median	One-pass max.	Optimal orderings
BM1 Arithmetic Intensity	19	11	2	9	13	15	54
BM2 Redundant Expressions	25	15	2	13	17	21	48
BM3 Dead-Code Heavy	15	3	2	3	4.5	6	240
BM4 Deep Copy Chain	10	3	2	3	5	10	240
BM5 Algebraic Identities	19	8	2	6	9	15	30
BM6 Input Arithmetic	13	9	7	7	9	9	120
BM7 Input Redundant Computation	16	9	7	7	9	9	120

TABLE 3. Variable-length search under matched candidate-evaluation budgets. Each method uses 750 evaluations per seed and 20 seeds.

Benchmark	GA median	Random median	Fixed-point cost	GA wins / 20	Ties / 20
BM1 Arithmetic Intensity	2	6	2	20	0
BM2 Redundant Expressions	2	7	2	20	0
BM3 Dead-Code Heavy	2	2	2	0	20
BM4 Deep Copy Chain	2	2	2	0	20
BM5 Algebraic Identities	2	2	2	0	20
BM6 Input Arithmetic	7	7	7	0	20
BM7 Input Redundant Computation	7	7	7	0	20

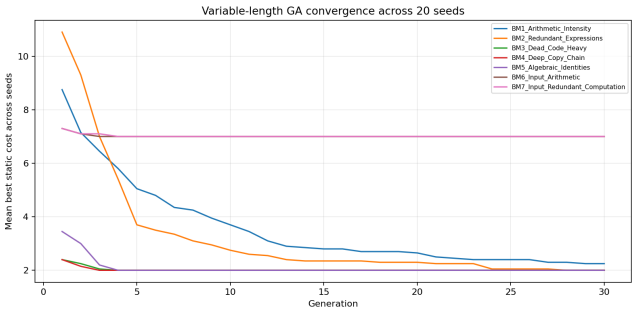


FIGURE 3. Mean best static cost per generation across 20 seeds for the variable-length GA. The curves summarize search behavior on the seven constructed benchmarks.

fixed-point baseline. On the constant-heavy benchmarks, removing CP, CF, or DCE can substantially increase the cost. On BM6 and BM7, removing AS increases the cost by six units, while removing CopyP increases it by two units. CSE removal has no effect on these seven examples under the fixed-point policy. This does not imply that CSE is generally unnecessary; it means only that the current programs and implementation do not expose a marginal benefit from it after the other passes have converged.

The ablation changes the available transformation set and therefore changes the optimization problem. The table should not be interpreted as an additive decomposition of each pass's effect because pass interactions are not independent. It is a controlled diagnostic of the chosen fixed-point pipeline on this benchmark suite.

E. WEIGHT SENSITIVITY

Table 5 reports the minimum Spearman rank correlation observed for each instruction category across the 0.75 and 1.25 perturbations and benchmarks. The minimum correlations range from 0.9797 for CONST to 0.9875 for COPY; INPUT and PRINT have correlation 1.0 because their counts are invariant across valid candidates in these programs. These high correlations indicate that the fixed-length ordering ranking is relatively stable to the tested local perturbations. They do not show that the weights correspond to actual processor cost. A score can be insensitive to small weight changes and still be a poor proxy for runtime or binary size.

F. SUMMARY OF EMPIRICAL FINDINGS

The experiments establish four limited findings. First, the six-pass one-use search is exactly enumerable and the fixed-length GA finds the exact optimum in all 140 benchmark-seed runs. Second, the variable-length GA beats random search on two constant-heavy programs and ties it on five under a 750-evaluation budget. Third, fixed-point execution yields identical final costs across all 720 one-use orderings for every benchmark in this suite. Fourth, the ablation and weight-sensitivity analyses provide evidence about the internal behavior of this specific IR and cost model. None of these results establishes a native speedup, executable-size reduction, or superiority over established compiler autotuning systems.

VII. DISCUSSION

A. WHY THE ORIGINAL GA RESULT WAS INSUFFICIENT

The 720-ordering result is a cautionary example. When the complete candidate space is small, exhaustive enumeration is more informative than a stochastic search because it proves the minimum under the stated objective. The GA's success rate of 100% across 20 seeds and median gap of zero are real measurements, but they are expected to be easy to obtain in a space where the optimum is frequent and the objective has many ties. The first-optimum generation is one in most runs, further indicating that the GA's search effort is not needed to discover these optima.

TABLE 4. Change in fixed-point cost after omitting one pass, relative to the full fixed-point pipeline. A zero means no cost change on that benchmark.

Benchmark	Without CP	Without CF	Without CopyP	Without CSE	Without AS	Without DCE
BM1	11	9	0	0	0	7
BM2	17	15	0	0	0	9
BM3	4	3	0	0	0	7
BM4	4	3	0	0	0	6
BM5	9	0	0	0	0	7
BM6	0	0	2	0	6	2
BM7	0	0	2	0	6	3

TABLE 5. Minimum Spearman rank correlation under local weight perturbations.

Instruction weight perturbed	Minimum correlation
CONST	0.9797
COPY	0.9875
BINOP	0.9815
INPUT	1.0000
PRINT	1.0000

The variable-length encoding enlarges the space and introduces pass selection, but it does not by itself make the benchmark suite realistic. It also creates equivalent sequences: omitting a pass that has no effect, repeating an idempotent pass, or moving passes that do not interact may yield the same output IR. Therefore, raw sequence-space size should not be confused with the number of semantically distinct outcomes. Deduplication and caching would reduce repeated evaluation cost, but would need to be reported consistently across comparators.

B. FIXED-POINT CLEANUP IS A STRONG BASELINE

The fixed-point results show that one-shot ordering sensitivity is not necessarily a property of the underlying pass set alone; it can be an artifact of the pass-application policy. Reapplying the fixed pipeline eliminates remaining opportunities for these seven programs. Because every ordering reaches the same cost in fixed-point mode, the benchmark suite cannot discriminate among orderings after convergence. This is a useful result, but it also shows why stronger benchmark programs are necessary: a pass library that converges every simple straight-line example to the same representation offers little basis for evaluating sequence search.

In a production compiler, fixed-point behavior depends on pass idempotence, convergence criteria, analysis invalidation, and the pass manager. The current helper re-runs the complete sequence until the IR is unchanged or the round limit is reached; it does not implement LLVM's pass manager and does not guarantee convergence for arbitrary transformations. In this experiment, all tested sequences converged within 20 rounds.

C. CORRECTNESS AND THE LIMITS OF FINITE TESTING

The semantic gate prevents candidates that change output on the declared test vectors from receiving positive fitness. This prevents a simple failure mode in which an unsafe rewrite obtains a low score by deleting or altering output-producing computation. However, testing is not proof. The input suite contains only five vectors for the input-driven programs and the interpreter models only straight-line operations. It cannot establish equivalence for all possible values, handle control-flow paths, or account for memory and side effects that are absent from the IR.

A production-quality optimizer would need stronger validation, such as translation validation, SMT-backed equivalence checking, differential testing against a reference compiler, or property-based generation of input cases. Each option has its own semantic assumptions and scaling cost. For this project, finite-test validation is a practical engineering safeguard and should be described as such.

D. STATIC COST VERSUS REAL PERFORMANCE

The instruction weights make the objective transparent and cheap. They are also a simplification. A real compiler can trade fewer IR instructions for increased register pressure, less vectorization, extra branches, or different memory behavior. Even a measured binary instruction count would not fully predict runtime across architectures. The local weight-sensitivity test addresses one narrow concern: whether small changes in weights change the ranking of the 720 fixed-length candidates. It does not calibrate the model to hardware.

A follow-up native experiment should translate the supported IR to C or LLVM IR, compile with a pinned compiler version, and measure both executable size and runtime. Runtime measurements would require fixed hardware, compiler flags, warm-up, repeated trials, and an appropriate statistic. If several objective measures conflict, a multi-objective Pareto analysis would be more defensible than a single hand-weighted scalar. Those measurements have not been performed and are not claimed in this revision.

E. RELATIONSHIP TO RECENT RL AND LLM METHODS

AutoPhase, dependence-graph approaches, DeCOS, and REASONING COMPILER use larger and more sophisticated decision spaces and evaluate objectives closer to their

target workloads. Their methods address search-space reduction, generalization, and context-sensitive proposals in settings where candidate evaluations are more expensive. The present implementation does not compete on those terms. Its contribution is to show that even a seemingly successful GA result can be misleading if the search space is enumerable, the baseline is weak, or the fitness objective is not linked to the desired outcome.

VIII. LIMITATIONS AND FUTURE WORK

The IR remains straight-line and does not support branches, loops, memory operations, function calls, or control-flow graphs. The seven benchmarks are hand-authored and small; two are input-driven but neither is a standard benchmark kernel. The static cost weights are not calibrated to native performance. Finite-test semantic validation is not formal equivalence. The 20-seed experiment characterizes one implementation and benchmark suite, not a population of compiler workloads. The matched budget counts candidate evaluations but does not equalize wall-clock overhead. The project repository URL supplied by the authors is https://github.com/Aditya-Rawat-11/Compiler-GA-Optimizer; the exact commit used for these experiments still needs to be recorded. A genuine screenshot of the Streamlit interface could not be captured in the packaging environment because Streamlit was unavailable. Author biographies, ORCID identifiers, and portrait photographs remain outstanding submission materials.

The next steps are limited and concrete. First, extend the IR with control-flow and loop constructs and implement CFG-aware data-flow passes, then translate standard kernels into the IR. Second, validate optimized code with a stronger equivalence or differential-testing approach. Third, add a native backend and measure runtime and executable size on pinned hardware, while retaining static cost as a separate diagnostic. Fourth, compare against more search heuristics under matched objective-call and wall-clock budgets. Fifth, add a genuine Streamlit screenshot and publish the code, benchmarks, raw CSVs, environment file, and commit hash in an author-controlled repository. These steps are future work, not completed experiments.

IX. CONCLUSION

This paper revises a small compiler optimization prototype in response to concerns about search-space size, benchmark realism, single-pass evaluation, and insufficient baselines. The fixed-length GA reaches the exact optimum in every one of 140 benchmark-seed runs, but the 720-ordering space is so small that exhaustive enumeration is the appropriate reference. A variable-length GA expands the candidate space to optional and repeated passes and beats random search on two constant-heavy benchmarks, while tying it on five and failing to improve on fixed-point cleanup. Fixed-point enumeration shows that every one-use ordering converges to the same cost on each benchmark, highlighting that one-pass rankings can be artifacts of the application policy. Pass ablation and local weight sensitivity add reproducible diagnostics, but the static score remains uncalibrated to runtime or code size. The evidence supports a transparent methodological case study, not a claim that genetic algorithms are generally superior or necessary for compiler optimization.

.

APPENDIX A BENCHMARK PROGRAMS

This appendix records the seven benchmark patterns so that the reported experiments can be understood independently of the interface. The exact tuple definitions are included in the project file `benchmarks.py`; the listings below use assignment notation for readability.

A. BM1: ARITHMETIC INTENSITY

```
a = 10
b = 20
t1 = a + b
t2 = t1 * 2
t3 = t1 + 0
t4 = t3 * 1
t5 = t4 + t2
PRINT t5
```

This program contains constants, arithmetic, a neutral addition, and multiplication by one. Its original static cost is 19. Its single-pass optimum is 9, while fixed-point cleanup reaches cost 2 because the final printed value can be represented directly as a literal in the supported IR.

B. BM2: REDUNDANT EXPRESSIONS

```
x = 4; y = 8
t1 = x + y
t2 = y + x
t3 = x * y
t4 = x * y
r1 = t1 + t2
r2 = t3 + t4
final = r1 + r2
PRINT final
```

This benchmark contains commutative arithmetic and repeated multiplication. The original cost is 25, the exact one-pass minimum is 13, and fixed-point cleanup reaches cost 2. Its low one-pass optimum still leaves opportunities because the pass set is applied only once.

C. BM3: DEAD-CODE HEAVY

```
base = 100; d1 = 50
dead_acc = base + d1
dead_val = dead_acc * 4
dead_copy = dead_val
active = 5
res = active + 10
PRINT res
```

The dead computation is not connected to PRINT. DCE can remove it once liveness is established. The original cost is 15, the exact one-pass minimum is 3, and fixed-point cleanup reaches cost 2 under the project's weighted instruction model.

D. BM4: DEEP COPY CHAIN

```
val = 42
c1 = val; c2 = c1
c3 = c2; c4 = c3
out = c4 + 8
PRINT out
```

This benchmark tests copy propagation and cleanup. Its original cost is 10, the exact one-pass minimum is 3, and fixed-point cleanup reaches cost 2.

E. BM5: ALGEBRAIC IDENTITIES

```
p = 12; q = 0
t1 = p + q
t2 = p - q
t3 = t1 * 0
t4 = t2 * 1
total = t3 + t4
PRINT total
```

This benchmark combines neutral and absorbing elements. The original cost is 19, the exact one-pass minimum is 6, and fixed-point cleanup reaches cost 2. The example illustrates why one-shot pass ordering can preserve expressions that a later iteration would simplify.

F. BM6: INPUT ARITHMETIC

```
INPUT x; INPUT y
sum = x + y
scaled = sum * 1
offset = scaled + 0
PRINT offset
```

The values of x and y are supplied by test cases, so constant propagation cannot simply replace them with literals. The original cost is 13, the exact one-pass minimum is 7, and fixed-point cleanup also reaches cost 7.

G. BM7: INPUT-DEPENDENT REDUNDANT COMPUTATION

```
INPUT x; INPUT y
t1 = x + y
t2 = y + x
t3 = t1 * 1
out = t3 + 0
PRINT out
```

This program combines inputs with commutative repeated expressions and algebraic identities. Its original cost is 16, the exact one-pass minimum is 7, and fixed-point cleanup reaches cost 7. It is still a short straight-line test, not a loop or a standard benchmark kernel.

APPENDIX B SEARCH ALGORITHMS AND EVALUATION PSEUDOCODE

The following pseudocode describes the candidate-level correctness gate used by the revised evaluation. It is language-independent and mirrors the implemented sequence: apply the pass list, validate the IR, run the reference interpreter on the declared test vectors, then calculate cost only if outputs are preserved.

```
score(program, sequence, test_inputs):
   candidate = copy(program)
   try:
      for pass in sequence:
         candidate = apply(pass, candidate)
         validate_ir(candidate)
      for inputs in test_inputs:
         if interpret(program, inputs) !=
              interpret(candidate, inputs):
            return INVALID, 0
      cost = weighted_instruction_cost(candidate)
      return cost, 1000 / (1 + cost)
   except ExecutionOrValidationError:
      return INVALID, 0
```

For fixed-length search, each chromosome is a permutation and order crossover preserves one occurrence of each pass. The mutation operator swaps two positions. For variable-length search, each chromosome is a list of pass identifiers; mutation may insert, delete, replace, or swap a gene, and the resulting sequence is clipped at the length cap. Selection uses tournament sampling and elitism preserves the best candidates found so far. The experiment script records the best cost by generation and the best pipeline for each seed.

The fixed-length search provides an exact reference because all 720 permutations can be enumerated. The variable-length sequence count, before accounting for semantic equivalence, is

$$\sum_{\ell=1}^{12} 6^{\ell} = \frac{6^{13} - 6}{5}. \tag{3}$$

This large number is an upper bound on raw strings, not on distinct optimized IRs or cost values. Many sequences are equivalent because passes may be idempotent or have no effect on a particular program. No exhaustive optimum is claimed for the variable-length space.

APPENDIX C SUPPLEMENTARY FIXED-LENGTH SEARCH RESULTS

Table 6 summarizes the 20-seed fixed-length results. Every benchmark has a 100% success rate, zero median optimality gap, and first reaches the exact optimum at generation 1 except BM5, where the median first-optimum generation is 1.5. These numbers characterize the configuration with population 25 and 30 generations. Because the optimum is often present in the initial random population, the first-generation result is not evidence of a sophisticated evolutionary improvement.

APPENDIX D SUPPLEMENTARY ABLATION AND WEIGHT ANALYSIS

The complete leave-one-pass-out experiment is stored in `results/pass_ablation_fixed_point.csv`. Table 4 in the main paper reports cost deltas from the full fixed-point pipeline. The key pattern is that CP, CF, and DCE matter for the constant-heavy programs, while AS is especially important for the input-driven programs. These findings should be read as benchmark-specific effects rather than

TABLE 6. Repeated fixed-length GA results over 20 seeds.

Benchmark	Runs	Success rate	Median cost	Median gap	Median first-optimum generation
BM1	20	100%	9	0	1.0
BM2	20	100%	13	0	1.0
BM3	20	100%	3	0	1.0
BM4	20	100%	3	0	1.0
BM5	20	100%	6	0	1.5
BM6	20	100%	7	0	1.0
BM7	20	100%	7	0	1.0

universal pass importance. The pass sequence is changed whenever a pass is removed, so interactions among the remaining passes can also change.

The complete local weight perturbation results are stored in `results/weight_sensitivity.csv`. The tested factors are 0.75, 1.0, and 1.25. Baseline correlations at factor 1.0 are exactly 1.0 by construction. The minimum correlation across benchmarks and perturbations remains above 0.979 for each instruction category. This supports a narrow statement of ranking stability within the 720-ordering space. It does not justify the selected weights as a model of execution time.

APPENDIX E REPRODUCIBILITY AND ARTIFACT STRUCTURE

The accompanying project contains the optimizer modules, pass implementations, benchmark definitions, unit tests, experiment harness, raw CSV files, and plotting script. The main experiment command is:

```
python research_experiments.py
```

The test suite can be run with:

```
python -m pytest -q
```

The result files include the seven-benchmark summary, fixed-length GA seed results, variable-length GA versus random search, exhaustive one-pass costs, exhaustive fixed-point costs, pass ablation, weight sensitivity, and per-generation convergence. The figure files are generated from these machine-readable outputs rather than transcribed manually. The experiment metadata records the seed set, pass names, cost weights, and the warning that the static score is not a runtime measurement.

The tests pass in the current packaging environment (19 tests). Streamlit was not installed in that environment, so a genuine browser capture of the interface could not be produced. The manuscript does not substitute a mock-up for a screenshot. Before submission, install the project's requirements, launch the application, capture the actual interface, and add the image. The repository should then be published by an author and the exact public URL and commit hash recorded; neither is invented here.

APPENDIX F SUBMISSION METADATA CHECKLIST

This revision uses the author order and all four contact emails supplied by the authors, with Aditya Rawat designated as corresponding author. Biographies, ORCID identifiers, and portrait photographs remain outstanding. No submission or publication DOI, received/revised/accepted dates, or other publisher-assigned metadata have been fabricated. The manuscript is compiled with the supplied IEEE Access class file.

APPENDIX G DETAILED COST-LANDSCAPE INTERPRETATION

The exhaustive cost CSV contains one row for every benchmark–ordering pair. For each benchmark, 720 candidate sequences are evaluated once in single-pass mode. The distributions in Fig. 2 show the count of orderings at each static-cost value, rather than a distribution of measured execution times. The minimum, median, and maximum values summarize the support and spread of this discrete objective. The cost range is six units for BM1, eight for BM2, three for BM3, seven for BM4, nine for BM5, and two for each input-driven program. The range is affected by the source pattern and the selected weights; it should not be interpreted as the range of performance that a compiler user would observe.

The number of optimal orderings is also important. BM1 has 54 minima, BM2 has 48, BM3 and BM4 each have 240, BM5 has 30, and BM6 and BM7 each have 120. In probability terms, an ideal uniform draw from the one-use permutation space has a 7.5% chance of finding a minimum for BM1, 6.7% for BM2, 33.3% for BM3 and BM4, 4.2% for BM5, and 16.7% for BM6 and BM7. These are exact fractions of the finite search space, not estimates from random trials. A 750-draw random search with replacement has many opportunities to sample a minimum when the optimal subset is large, which helps explain why the original random baseline was unexpectedly strong.

The fixed-point enumeration provides a complementary landscape. All 720 orderings produce the same fixed-point cost for each benchmark. For BM1–BM5, that cost is 2; for BM6 and BM7, it is 7. The fixed-point landscape has zero cost variance and no useful rank ordering. In this setting, reporting a Spearman correlation between single-pass and fixed-point cost rankings is mathematically undefined because the fixed-point ranks are all tied. This is not a missing experiment: it is the observed outcome. It does, however, limit what this benchmark suite can tell us about order-sensitive search after convergence.

The distinction between a one-pass and fixed-point land-

TABLE 7. Landscape summary for single-pass and fixed-point execution.

Benchmark	Range	Optimal fraction	FP cost	FP ties
BM1	6	7.5%	2	720/720
BM2	8	6.7%	2	720/720
BM3	3	33.3%	2	720/720
BM4	7	33.3%	2	720/720
BM5	9	4.2%	2	720/720
BM6	2	16.7%	7	720/720
BM7	2	16.7%	7	720/720

scape must be preserved in future work. The fixed-length GA's optimality gap is defined against the exact one-pass optimum, not against a production compiler's output. The variable-length GA's best cost is compared with random search under a finite evaluation budget, not with an exact optimum. Fixed-point costs are a deterministic baseline, but their equality across all sequences means the benchmark suite cannot distinguish which variable-length search is preferable when candidates are subsequently iterated to convergence. A larger suite with non-idempotent, control-flow-aware passes may reveal a different result; no such result is assumed here.

APPENDIX H INPUT CASES AND CORRECTNESS GATE DETAILS

The semantic gate uses the same interpreter for original and candidate IR, so the comparison checks the observable outputs under the implementation's own defined semantics. For constant-only benchmarks, the harness evaluates the empty input tuple. For input-driven programs, the harness provides five input vectors. BM6 and BM7 each declare two inputs, x and y, so the tested vectors are $(0, 0)$, $(1, 1)$, $(-3, 7)$, $(2, 5)$, and $(11, -4)$. These vectors include zeros, positive values, a negative value, and mixed signs. They do not exhaust the numeric domain and do not cover every possible divisor or arithmetic edge case.

For each vector, the interpreter evaluates instructions in sequence, stores values in a variable map, and records PRINT values. A missing input, use of an undefined variable, unsupported opcode, or division by zero raises an execution error. Such a candidate is rejected by the fitness evaluator. If the number of outputs differs or any output differs beyond the interpreter's numeric tolerance, the candidate is rejected. The cost is computed only after the candidate passes these checks. This prevents a low-cost candidate from winning merely because it removes a required output or changes a tested result.

The check is a finite-test validation gate, not formal translation validation. It is possible for a candidate to agree on the five selected input vectors and differ on an untested vector. For example, an unsafe rewrite of division could fail only when the denominator takes a value not represented in the test cases. The algebraic simplification pass therefore uses conservative rules for variable denominators, and the interpreter raises on division by zero. Even with those safeguards, the current test strategy cannot certify behavior for arbitrary input values.

The scope is further limited by the IR grammar. Since the program is linear, every instruction is executed exactly once and the interpreter has no control-flow state. There are no branch conditions, loop induction variables, heap objects, pointers, calls, or memory aliases. A control-flow-capable extension would need basic blocks and edges, a control-flow graph, dominance and liveness analyses, and semantics for side effects. Merely adding a BRANCH opcode without updating the passes and interpreter would not constitute a correct extension.

For a stronger evaluation, the next version should combine generated test inputs with property-based testing and differential execution against a trusted reference. Where possible, translation validation could compare original and optimized IR using an SMT solver or a compiler-supported equivalence checker. Those methods require explicit integer or floating-point semantics and may be expensive for richer programs. The current paper therefore uses the narrower phrase "output preservation on supplied test cases" rather than claiming semantic equivalence.

TABLE 8. Finite test vectors for the input-driven benchmarks.

Case	x	y
1	0	0
2	1	1
3	-3	7
4	2	5
5	11	-4

APPENDIX I EVALUATION BUDGET AND COMPLEXITY NOTES

The fixed-length GA uses a population of 25 and 30 generations, so its nominal budget is 750 candidate evaluations per seed. The variable-length GA uses the same population and generation count. Random search also evaluates 750 candidate sequences per seed. All methods use the same pass implementations, static cost function, and semantic gate. This design matches the number of candidate evaluations for variable-length GA and random search, but it does not equalize CPU time: the GA spends additional effort on parent selection, crossover, and mutation. Since no native compilation or application execution occurs inside candidate evaluation, the dominant cost is Python-level transformation and interpretation.

The fixed-length exhaustive reference evaluates exactly 720 distinct permutations. It therefore evaluates fewer candidates than the 750-call search budget and avoids duplicates by construction. The experiment harness does not claim a matched budget between exhaustive enumeration and GA; instead, enumeration is used as an exact reference because the space is small. This distinction is important because candidate count and unique candidate coverage are not identical. A stochastic algorithm can repeatedly evaluate the same sequence, while exhaustive enumeration visits each permutation once.

Let K be the number of passes, L the maximum sequence length, N the population size, and G the number of generations. Fixed-length exhaustive enumeration has $K!$ candidates when each pass is used once. The number of raw variable-length sequences from length 1 to L is $\sum_{\ell=1}^{L} K^{\ell}$. The variable-length GA's nominal candidate count is NG. If applying a pass to an IR of n instructions costs $T_p(n)$, then one candidate's cost is the sum of the pass costs along its sequence, plus validation and interpretation. A rough upper-bound expression for a length-L candidate is $O(L\,T_{\max}(n) + T_{\text{validate}}(n) + T_{\text{interpret}}(n))$; this is a schematic bound rather than a formal complexity result for every pass implementation.

Fixed-point mode adds repeated pipeline application. If the sequence converges after r rounds, the candidate cost is approximately r times the one-pass pipeline cost, plus the final equality comparison. The implementation stops after at most 20 rounds. In the experiments reported here, every tested ordering converges before the guard. This observed convergence is specific to the present passes and benchmark suite; it should not be generalized to arbitrary transformations, especially if a future pass can oscillate or introduce non-monotone rewrites.

A useful next engineering improvement would be memoization keyed by the benchmark identifier, exact sequence, policy, and cost weights. Cache entries should also include validation status and the resulting IR. The policy and weights must be part of the key because the same sequence can produce different outcomes under single-pass and fixed-point modes or under different cost models. Deduplicating candidates would reduce wasted transformation work, but any future GA/random comparison should report both attempted and unique evaluations and should keep the budget definition consistent.

APPENDIX J THREATS TO VALIDITY

A. CONSTRUCT VALIDITY

The principal construct is optimization quality. It is measured using a weighted instruction count rather than native runtime, cycle counts, energy, or binary size. The selected weights encode a transparent project preference but are not calibrated to a target architecture. The weight-sensitivity analysis establishes only that local perturbations usually preserve rankings within the small one-pass space. It does not establish predictive validity. The static score can rank two programs differently from a real compiler because it ignores instruction latency, memory behavior, vectorization, register allocation, and control-flow frequency.

B. INTERNAL VALIDITY

The outcomes depend on pass implementation details, the sequence representation, the fixed-point convergence cap, and the finite input vectors. The semantic gate catches tested output mismatches and execution errors, but it cannot prove equivalence. The experiment uses 20 seeds, which is more informative than a single seed but still a modest sample of stochastic behavior. Since many fixed-length orderings have the same cost, the success rate can be high even if the search does not meaningfully improve the population. The first generation already contains the optimum in most runs, so the reported convergence generation must be interpreted alongside the exact landscape statistics.

The GA and random-search comparison is matched by candidate-evaluation count, not elapsed time or unique sequence count. The variable-length GA uses tournament selection, crossover, mutation, and elitism, whereas random search draws independently from the sequence generator. The methods are therefore compared on best cost under a fixed number of attempts, not on computational efficiency. The fixed-point baseline is a deterministic comparator and is not a stochastic search method.

C. EXTERNAL VALIDITY

All seven programs are hand-authored, small, straight-line examples. Two accept external scalar values, but neither has loops or branches. No standard suite such as SPEC, PolyBench, or CBench is translated into the IR. The results therefore cannot be extrapolated to general-purpose programs or industrial compilers. The pass library contains six simple transformations and omits many production passes, analyses, and target-dependent optimizations. A broader study must include programs that exercise control flow, memory, and loops, with passes that are correct for those constructs.

D. REPRODUCIBILITY VALIDITY

The code, experiment script, CSV outputs, and figures are included in the working package, and the unit tests pass in the build environment. The authors supplied the public repository URL https://github.com/Aditya-Rawat-11/Compiler-GA-Optimizer; the exact commit used for the reported experiments has not yet been verified or recorded. The Streamlit UI could not be launched in the build environment because the dependency was unavailable, so a genuine screenshot is still outstanding. The supplied IEEE Access class file is included in this package and used to compile this revision. Author biographies, ORCID identifiers, and portrait photographs remain to be supplied by the authors.

E. MITIGATION PLAN

These threats are not addressed by adding repeated caveats to every result. Instead, they define the limits of the claims. The current manuscript reports static costs as static costs, finite testing as finite testing, and a custom test suite as a custom test suite. The follow-up evaluation should use standard benchmark kernels, stronger semantic validation, native runtime and size measurements, a pinned compiler and environment, a public versioned repository, and a broader comparator set. Until then, the paper is a reproducible small-IR case study rather than a validated production compiler-optimization method.

APPENDIX K ARTIFACT INVENTORY AND REPRODUCTION CHECKLIST

The working artifact package is organized to separate implementation, experiment outputs, figures, and manuscript sources. The main code is under the project root; pass implementations are under `passes/`; benchmark definitions are in `benchmarks.py`; the GA implementations are under `ga/`; fixed-point helpers are in `pipeline_modes.py`; and finite-test interpretation is in `semantics.py`. The experiment entry point is `research_experiments.py`. Its output directory contains raw CSVs and JSON metadata.

The following files are especially important for checking the numerical claims:

- `benchmark_summary.csv`: baseline, fixed-point, exact single-pass, and repeated-seed summary costs.
- `ga_20_seed_results.csv`: fixed-length GA outcome for each benchmark and seed.
- `variable_ga_vs_random_20_seeds.csv`: paired variable-length GA and random-search results.
- `exhaustive_ordering_costs.csv`: all 720 single-pass ordering costs per benchmark.
- `fixed_point_ordering_costs.csv`: all 720 orderings evaluated in fixed-point mode.
- `pass_ablation_fixed_point.csv`: leave-one-pass-out costs.
- `weight_sensitivity.csv`: rank correlations under local weight perturbations.
- `ga_convergence_by_seed.csv`: per-generation search history.

The expected reproduction workflow is to create a clean Python environment, install dependencies from the project requirements file, run the unit tests, run the experiment script, and compare generated CSV summaries with the supplied versions. Figures should be regenerated from the CSVs rather than edited manually. The experiment script records 20 seeds and uses a fixed pass registry. The environment version should be recorded when results are regenerated; timing measurements are not emphasized because they are small and are not the optimization objective.

The authors supplied the repository URL https://github.com/Aditya-Rawat-11/Compiler-GA-Optimizer. The repository could not be independently fetched in this build environment, and no commit hash/tag was supplied. Before submission, verify that the repository contains the exact code and result files used here and record the commit that corresponds to the reported experiments.

The paper package includes architecture, convergence, and exhaustive-cost-distribution figures. It does not include a Streamlit screenshot because the app could not be launched in the packaging environment. A screenshot must be captured from the actual running application and should show the interface state used for the paper. A designed mock-up or static diagram is not an acceptable substitute for a screenshot. The same principle applies to author photographs and biography text: the authors should provide them directly rather than have them inferred or fabricated.

APPENDIX L RECOMMENDATIONS FOR A STRONGER COMPARATIVE STUDY

The next comparative study should treat search-space representation, candidate-evaluation cost, and optimization quality as separate experimental variables. A larger sequence space can be constructed by adding passes, allowing optional and repeated passes, and introducing prerequisites only when those prerequisites are grounded in pass semantics. The search space should be specified mathematically and the maximum sequence length justified before results are inspected. The experiment should report the raw number of candidate strings, the number of distinct sequences sampled, the number of distinct optimized IRs, and the number of distinct objective values. These quantities answer different questions and should not be collapsed into one search-space size.

A fair comparison should use a shared candidate evaluator and shared correctness policy. GA, random search, mutation-only local search, and a fixed heuristic should receive the same number of objective calls. A second comparison should use a wall-clock budget, since genetic operators and duplicate evaluations contribute overhead. For each seed, the methods should use the same benchmark and input vectors; paired reporting then makes it possible to distinguish differences due to a method from differences due to a benchmark. If an exact optimum is available, report success rate, median and interquartile optimality gap, and first generation at optimum. If the space is too large to enumerate, do not label the best observed candidate as the optimum; call it the best observed cost under the specified budget.

Benchmarks should be selected to expose different pass interactions. Straight-line examples are useful for unit tests, but a research benchmark suite should include control-flow joins, loop-carried dependencies, memory accesses, and computations that cannot be fully constant-folded. Each added IR construct requires a semantics definition, interpreter support, validator rules, and pass-specific correctness tests. Standard kernels should be translated in a documented way, and the translated program's outputs should be compared with a trusted reference over multiple inputs. Benchmarks used to tune pass weights should be separated from benchmarks used to report final results to reduce the risk of tailoring the objective to the evaluation set.

The cost model should be validated independently from the search algorithm. Candidate ranking can be compared against compiled runtime, code size, or hardware counters using rank correlation and top-k overlap. If the relation is weak, the model should be revised or the paper should focus on static IR simplification rather than performance. If several outcomes matter, such as runtime and code size, the experiment should report Pareto fronts or a predeclared scalarization. The weight-sensitivity test in this paper is a useful first diagnostic but is not a substitute for external calibration.

Finally, reproducibility requires a tagged source revision, pinned dependencies, benchmark inputs, test outputs, raw

result files, plotting scripts, and a complete environment description. The supplied public repository should include the exact manuscript figures and a script that regenerates the tables; the precise commit corresponding to this manuscript should be recorded. Author-supplied biographies, photographs, emails, and the final IEEE Access template should be checked separately from the technical experiment. This protocol is intended as a concrete follow-up plan; none of the unperformed runtime, standard-benchmark, or public-release tasks is claimed as completed in this paper.

ACKNOWLEDGMENT

The authors used OpenAI ChatGPT to assist with language editing and restructuring of portions of this revised manuscript, specifically the Abstract, Introduction, discussion of results, limitations, and reproducibility-related text. The tool was not used to conduct the reported experiments or establish the reported measurements. The authors are responsible for checking the manuscript, validating the technical claims and references, and approving the final text. The use of this tool is disclosed in accordance with IEEE guidance [16].

REFERENCES

[1] K. D. Cooper, P. J. Schielke, and D. Subramanian, "Optimizing for reduced code space using genetic algorithms," in Proc. ACM SIGPLAN Workshop Languages, Compilers, and Tools for Embedded Systems, 1999, pp. 1–9, doi: 10.1145/314403.314414.

[2] Z. Pan and R. Eigenmann, "Fast and effective orchestration of compiler optimizations for automatic performance tuning," in Proc. Int. Symp. Code Generation and Optimization (CGO), 2006, pp. 319–332, doi: 10.1109/CGO.2006.38.

[3] K. Hoste and L. Eeckhout, "COLE: Compiler optimization level exploration," in Proc. Int. Symp. Code Generation and Optimization (CGO), 2008, pp. 165–174, doi: 10.1145/1356058.1356080.

[4] A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, and C. Silvano, "A survey on compiler autotuning using machine learning," ACM Comput. Surv., vol. 51, no. 5, Art. no. 96, 2018, doi: 10.1145/3197978.

[5] G. Fursin et al., "Milepost GCC: Machine learning enabled self-tuning compiler," Int. J. Parallel Program., vol. 39, pp. 296–327, 2011, doi: 10.1007/s10766-010-0161-2.

[6] Y. Liang et al., "Learning compiler pass orders using coreset and normalized value prediction," in Proc. 40th Int. Conf. Machine Learning, Proc. Mach. Learn. Res., vol. 202, 2023. Available: https://proceedings.mlr.press/v202/liang23a.html.

[7] H. Peeler, S. S. Li, A. N. Sloss, K. Reid, Y. Yuan, and W. Banzhaf, "Optimizing LLVM pass sequences with Shackleton: A linear genetic programming framework," in Proc. Genetic and Evolutionary Computation Conf. Companion (GECCO), 2022, pp. 578–581, doi: 10.1145/3520304.3528945.

[8] A. Haj-Ali, Q. Huang, J. Xiang, W. Moses, K. Asanović, J. Wawrzynek, and I. Stoica, "AutoPhase: Juggling HLS phase orderings in random forests with deep reinforcement learning," in Proc. Conf. Machine Learning and Systems, vol. 2, 2020. Available: https://proceedings.mlsys.org/paper_files/paper/2020/hash/5b47430e24a5a1f9fe21f0e8eb814131-Abstract.html.

[9] J. Liu et al., "Efficient compiler optimization by modeling passes dependence," CCF Trans. High Perform. Comput., vol. 6, pp. 588–607, 2024, doi: 10.1007/s42514-024-00197-9.

[10] T. Cui, P.-C. Yew, S. McCamant, and A. Zhai, "DeCOS: Data-efficient reinforcement learning for compiler optimization selection ignited by LLM," in Proc. ACM Int. Conf. Supercomputing (ICS), 2025, pp. 943–958, doi: 10.1145/3721145.3725765.

[11] A. S. Tang, C. Priebe, R. Mahapatra, L. Qin, and H. Esmaeilzadeh, "Reasoning compiler: LLM-guided optimizations for efficient model serving," in Advances in Neural Information Processing Systems, vol. 38, 2025, doi: 10.52202/085713-3565.

[12] M. R. Jantz and P. A. Kulkarni, "Performance potential of optimization phase selection during dynamic JIT compilation," in Proc. 9th ACM SIGPLAN/SIGOPS Int. Conf. Virtual Execution Environments (VEE), 2013, pp. 131–142, doi: 10.1145/2451512.2451539.

[13] N. L. Queiroz Junior, A. F. da Silva, and L. G. A. Rodriguez, "Finding effective compiler optimization sequences: A hybrid approach," Computing and Informatics, vol. 39, no. 6, pp. 1117–1147, 2020, doi: https://doi.org/10.31577/cai_2020_6_1117.

[14] C. Cummins et al., "CompilerGym: Robust, performant compiler optimization environments for AI research," arXiv:2109.08267, 2021. Available: https://arxiv.org/abs/2109.08267.

[15] K. D. Cooper, D. Subramanian, and L. Torczon, "Adaptive optimizing compilers for the 21st century, J. Supercomput., vol. 23, no. 1, pp. 7–22, 2002, doi: 10.1023/A:1015729001611.

[16] OpenAI, ChatGPT, online resource. Available: https://chatgpt.com/. Accessed: Oct. 28, 2026.

AUTHOR BIOGRAPHIES

Aditya Rawat – Author biography to be supplied and approved by the author before submission.

Abhishek Kumar – Author biography to be supplied and approved by the author before submission.

Tamanna Thakur – Author biography to be supplied and approved by the author before submission.

Sanvi Tiwari – Author biography to be supplied and approved by the author before submission.

Submission note: These placeholders are intentional and must be replaced with factual biographies before submission. ORCID identifiers and author photographs, if required by the final submission workflow, should be added only when supplied by the authors.